

Relatório Técnico — Estudo Dirigido 2: Padrões de Projeto

Aluno: Pedro Gomes Sampaio **Disciplina:** Programação Orientada a Objetos **Entrega:** 28/07/2026

Cenário de Referência

Os exemplos práticos deste relatório partem do mesmo cenário utilizado no Estudo Dirigido 1: a **autorização de procedimentos por plano de saúde**.

O sistema decide, em sequência, se um procedimento solicitado por um beneficiário será autorizado ou negado, avaliando cobertura, carência, autorização prévia e calculando a coparticipação devida. É sobre a estrutura interna desse sistema — as decisões de design que o sustentam — que os padrões de projeto abaixo incidem.

1. Padrões Criacionais

1.1 Conceito

Padrões criacionais resolvem um problema recorrente de acoplamento: quem usa um objeto não deveria precisar conhecer qual classe concreta está sendo instanciada, nem os detalhes de como esse objeto é construído. Ao encapsular a lógica de criação, esses padrões permitem trocar implementações sem alterar o código que consome o objeto.

Características gerais:

- Separam a decisão de “qual classe criar” de quem utiliza o objeto.
- Controlam o ciclo de vida das instâncias (quantidade, momento de criação, descarte).
- Facilitam a substituição de implementações sem impacto nos clientes.

Aplicabilidade: Use padrões criacionais quando a criação de um objeto é complexa, quando o chamador não deve depender de uma classe concreta específica, ou quando o sistema precisa garantir restrições sobre o número de instâncias existentes.

1.2 Padrão Aplicado: Singleton

Problema

No sistema de autorização, os repositórios (`RepositorioBeneficiario`, `RepositorioPlano`, `RepositorioProcedimento`) mantêm os dados em memória durante a execução. Se duas partes do código instanciassem `RepositorioBeneficiario` independentemente, cada instância teria sua própria lista — beneficiários adicionados em uma não apareceriam na outra, gerando inconsistência de estado.

```
// Problema: nada impede múltiplas instâncias
RepositorioBeneficiario rep1 = new RepositorioBeneficiario();
RepositorioBeneficiario rep2 = new RepositorioBeneficiario();
rep1.adicionar(new Beneficiario("Pedro", ...));
rep2.listar(); // retorna lista vazia – rep2 não conhece Pedro
```

Solução

O **Singleton** garante que uma classe possua apenas uma instância no sistema e fornece um ponto de acesso global a ela. A classe assume o controle da própria criação: o construtor é privado e a instância é exposta por um método estático.

Implementação

```
public class RepositorioBeneficiario {

    // instância única criada na carga da classe (thread-safe sem
    sincronização)
    private static final RepositorioBeneficiario INSTANCE =
        new RepositorioBeneficiario();

    // construtor privado: impede new RepositorioBeneficiario() fora
    da classe
    private RepositorioBeneficiario() {}

    public static RepositorioBeneficiario getInstance() {
        return INSTANCE;
    }

    // --- mesmos métodos do repositório original ---

    private final List<Beneficiario> list = new ArrayList<>();

    public void adicionar(Beneficiario beneficiario) {
        list.add(beneficiario);
    }

    public List<Beneficiario> listar() {
        return Collections.unmodifiableList(list);
    }
}
```

```

    public Beneficiario buscar(String nome) {
        for (Beneficiario b : list) {
            if (nome.equals(b.getNome())) return b;
        }
        throw new BeneficiarioNaoEncontradoException(
            "Beneficiário nomeado por " + nome + " não encontrado");
    }
}

```

Com o Singleton aplicado, qualquer ponto do sistema que precise do repositório obtém a mesma instância via `RepositorioBeneficiario.getInstance()`, eliminando a possibilidade de inconsistência por duplicação.

2. Padrões Estruturais

2.1 Conceito

Padrões estruturais tratam de como objetos e classes são compostos para formar estruturas maiores. O mecanismo central é a **composição**: montar comportamento novo combinando objetos, sem herança e sem modificar o que já existe.

Características gerais:

- Organizam classes e objetos em estruturas que sejam fáceis de estender.
- Usam composição em vez de herança para adicionar responsabilidades.
- Permitem que interfaces incompatíveis colaborem, ou que comportamentos sejam empilhados de forma transparente ao chamador.

Aplicabilidade: Use padrões estruturais quando precisar adicionar comportamento a objetos sem alterar sua classe, quando interfaces existentes forem incompatíveis entre si, ou quando a composição de funcionalidades precisar ser dinâmica e intercambiável.

2.2 Padrão Aplicado: Decorator

Problema

O `ServicoAutorizacao` realiza a lógica central de decisão. Em determinado momento da evolução do sistema, surge a necessidade de registrar em log cada solicitação de autorização — quem pediu, qual procedimento e qual foi o resultado. Modificar diretamente `ServicoAutorizacao` viola o princípio de que classes devem estar abertas para extensão, mas fechadas para modificação. Além disso, criar uma subclasse para cada variação de comportamento adicional (log, métricas, auditoria) explode a hierarquia.

```

// Sem Decorator: modificar ServicoAutorizacao para adicionar log
public ResultadoAutorizacao autorizar(...) {
    System.out.println("[LOG] Solicitação recebida..."); // não
    deveria estar aqui
    // ... lógica de negócio misturada com infraestrutura
}

```

Solução

O **Decorator** envolve (*wraps*) um objeto com uma camada que adiciona comportamento, sem que o chamador perceba a diferença — ambos expõem a mesma interface. Comportamentos podem ser empilhados: log em volta de métricas em volta do serviço real.

Implementação

Passo 1 — extrair a interface:

```
public interface IServicoAutorizacao {
    ResultadoAutorizacao autorizar(
        Beneficiario beneficiario,
        Procedimento procedimento,
        ContextoAtendimento contexto);
}
```

Passo 2 — o componente concreto implementa a interface (sem alteração da lógica):

```
public class ServicoAutorizacao implements IServicoAutorizacao {
    @Override
    public ResultadoAutorizacao autorizar(
        Beneficiario beneficiario,
        Procedimento procedimento,
        ContextoAtendimento contexto) {
        // lógica de negócio inalterada
        if (!beneficiario.getPlano().cobre(procedimento)) {
            return new ResultadoAutorizacao(Decisao.NEGADO_COBERTURA,
                "Plano não cobre");
        }
        // ... demais verificações
        return new ResultadoAutorizacao(Decisao.AUTORIZADO,
            "Autorizado",
            beneficiario.getPlano().calcularCopart(procedimento));
    }
}
```

Passo 3 — o Decorator envolve qualquer IServicoAutorizacao:

```
public class ServicoAutorizacaoComLog implements IServicoAutorizacao {

    private final IServicoAutorizacao delegado;

    public ServicoAutorizacaoComLog(IServicoAutorizacao delegado) {
        this.delegado = delegado;
    }

    @Override
    public ResultadoAutorizacao autorizar(
        Beneficiario beneficiario,
        Procedimento procedimento,
```

```

ContextoAtendimento contexto) {

    System.out.printf("[LOG] Solicitação: beneficiário=%s,
procedimento=%s%n",
        beneficiario.getNome(), procedimento.getCodigo());

    ResultadoAutorizacao resultado =
        delegado.autorizar(beneficiario, procedimento, contexto);

    System.out.printf("[LOG] Resultado: %s - %s%n",
        resultado.getDecisao(), resultado.getMensagem());

    return resultado;
}
}

```

Uso no ponto de composição (Main):

```

IServicoAutorizacao servico =
    new ServicoAutorizacaoComLog(new ServicoAutorizacao());

```

O chamador recebe um `IServicoAutorizacao` e chama `.autorizar()` normalmente. A camada de log é invisível para ele; pode ser removida ou substituída por outra decoração sem alterar nenhuma outra classe.

3. Padrões Comportamentais

3.1 Conceito

Padrões comportamentais tratam de como objetos distribuem responsabilidades e colaboram para cumprir o propósito do sistema. O foco está em **quem faz o quê** e em como variações de comportamento podem ser encapsuladas de forma intercambiável, sem que o chamador precise conhecer os detalhes de cada variante.

Características gerais:

- Encapsulam algoritmos ou comportamentos que variam, isolando-os do código que os usa.
- Definem protocolos de comunicação entre objetos de forma que o acoplamento seja mínimo.
- Permitem trocar comportamentos em tempo de execução sem alterar os clientes.

Aplicabilidade: Use padrões comportamentais quando um comportamento precisa variar independentemente de quem o usa, quando o sistema precisa selecionar entre algoritmos em tempo de execução, ou quando a responsabilidade por uma decisão deve estar concentrada em um objeto especializado.

3.2 Padrão Aplicado: Strategy

Problema

Planos de saúde podem adotar políticas de coparticipação distintas: percentual sobre o valor do procedimento, valor fixo por tipo de procedimento ou isenção total. Se a lógica de cálculo ficasse dentro de Plano com if/else ou switch, qualquer novo tipo de política exigiria modificar a classe — e o cálculo correto para cada caso ficaria espalhado e difícil de testar isoladamente.

```
// Sem Strategy: lógica de cálculo dentro de Plano
public double calcularCopart(Procedimento p) {
    if (tipoPolitica.equals("percentual")) {
        return p.getValorBase() * percentual;
    } else if (tipoPolitica.equals("fixo")) {
        return tabelaFixa.get(p.getTipo());
    } else {
        return 0.0;
    }
}
```

Solução

O **Strategy** define uma família de algoritmos, encapsula cada um em um objeto separado e os torna intercambiáveis. Plano delega o cálculo para a política que recebeu — e não sabe, nem precisa saber, qual variante está em uso.

Implementação

A interface Strategy:

```
public interface PoliticaCoparticipacao {
    double calcular(Procedimento p);
}
```

Estratégias concretas:

```
// Política percentual: cobra percentual do valor base
public class CoparticipacaoPercentual implements
    PoliticaCoparticipacao {
    private final double percentual;

    public CoparticipacaoPercentual(double percentual) {
        this.percentual = percentual;
    }

    @Override
    public double calcular(Procedimento p) {
        return p.getValorBase() * percentual;
    }
}
```

```

// Política isenta: sem cobrança
public class SemCoparticipacao implements PoliticaCoparticipacao {
    @Override
    public double calcular(Procedimento p) {
        return 0.0;
    }
}

// Política por tipo: valor fixo conforme categoria do procedimento
public class CoparticipacaoFixaPorTipo implements
    PoliticaCoparticipacao {
    private final Map<TipoProcedimento, Double> tabela;

    public CoparticipacaoFixaPorTipo(Map<TipoProcedimento, Double>
        tabela) {
        this.tabela = tabela;
    }

    @Override
    public double calcular(Procedimento p) {
        return tabela.getOrDefault(p.getTipo(), 0.0);
    }
}

```

O contexto (Plano) delega para a política:

```

public class Plano {
    private final PoliticaCoparticipacao politica;
    // ...

    public double calcularCopart(Procedimento p) {
        return politica.calcular(p); // não sabe qual Strategy está
        aqui
    }
}

```

Composição no ponto de criação:

```

// Plano ambulatorial: sem coparticipação
Plano planoAmbulatorial = new Plano("Unimed Ambulatorial",
    new Ambulatorial(...), new SemCoparticipacao(), 300, 180);

// Plano hospitalar: 20% sobre o valor base
Plano planoHospitalar = new Plano("Unimed Hospitalar Plus",
    new HospitalarComObstetricia(...), new
    CoparticipacaoPercentual(0.2), 300, 180);

```

A política pode ser trocada sem alterar Plano, ServicoAutorizacao ou qualquer outro componente. Adicionar uma quarta política exige apenas uma nova classe que implemente PoliticaCoparticipacao.

Referências

GAMMA, E. et al. *Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos*. Porto Alegre: Bookman, 2000.

Refactoring.Guru. *Design Patterns*. Disponível em: <https://refactoring.guru/pt-br/design-patterns>. Acesso em: jul. 2026.